



Association Analysis II

Lecture 10

肖升生 博士

Dr. Shengsheng Xiao

Email: Xiao.shengsheng@shufe.edu.cn



Outline

- FP-Growth
- Maximal and Closed Frequent Item Set
- Variants of Association Rule Mining
 - Mining with Multiple Minimum Supports
 - Mining Class Association Rules
 - Multi-level Association Rule
 - Sequential Pattern Mining



Outline

- **FP-Growth**
- Maximal and Closed Frequent Item Set
- Variants of Association Rule Mining
 - Mining with Multiple Minimum Supports
 - Mining Class Association Rules
 - Multi-level Association Rule
 - Sequential Pattern Mining



Mining Association Rules

- Two-step approach:
 1. Frequent Itemset Generation
 - Generate all item sets whose support $\geq$ minsup
 2. Rule Generation
 - Generate high confidence rules from each frequent item set, where each rule is a binary partitioning of a frequent item set



Bottleneck of Apriori

- Mining long patterns needs many passes of scanning and generates lots of candidates
- Bottleneck: **candidate-generation-and-test**
- Can we avoid **candidate generation**?
- Some new data structure can help!



FP-growth Algorithm

- Use a compressed representation of the database
- **FP-tree**

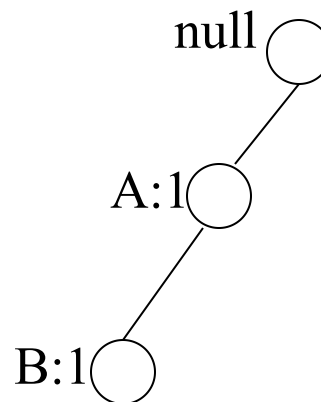
FP-tree construction:

- Scan DB once, find frequent 1-itemset (single item pattern)
- Sort frequent items in frequency descending order, F-list
- Scan DB again, construct FP-tree

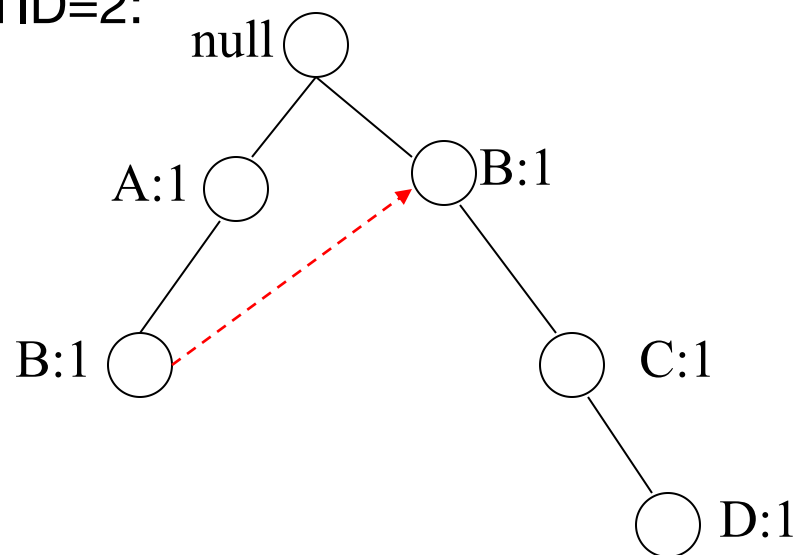
FP-tree construction

TID	Items
1	{A,B}
2	{B,C,D}
3	{A,C,D,E}
4	{A,D,E}
5	{A,B,C}
6	{A,B,C,D}
7	{A}
8	{A,B,C}
9	{A,B,D}
10	{B,C,E}

After reading TID=1:



After reading TID=2:



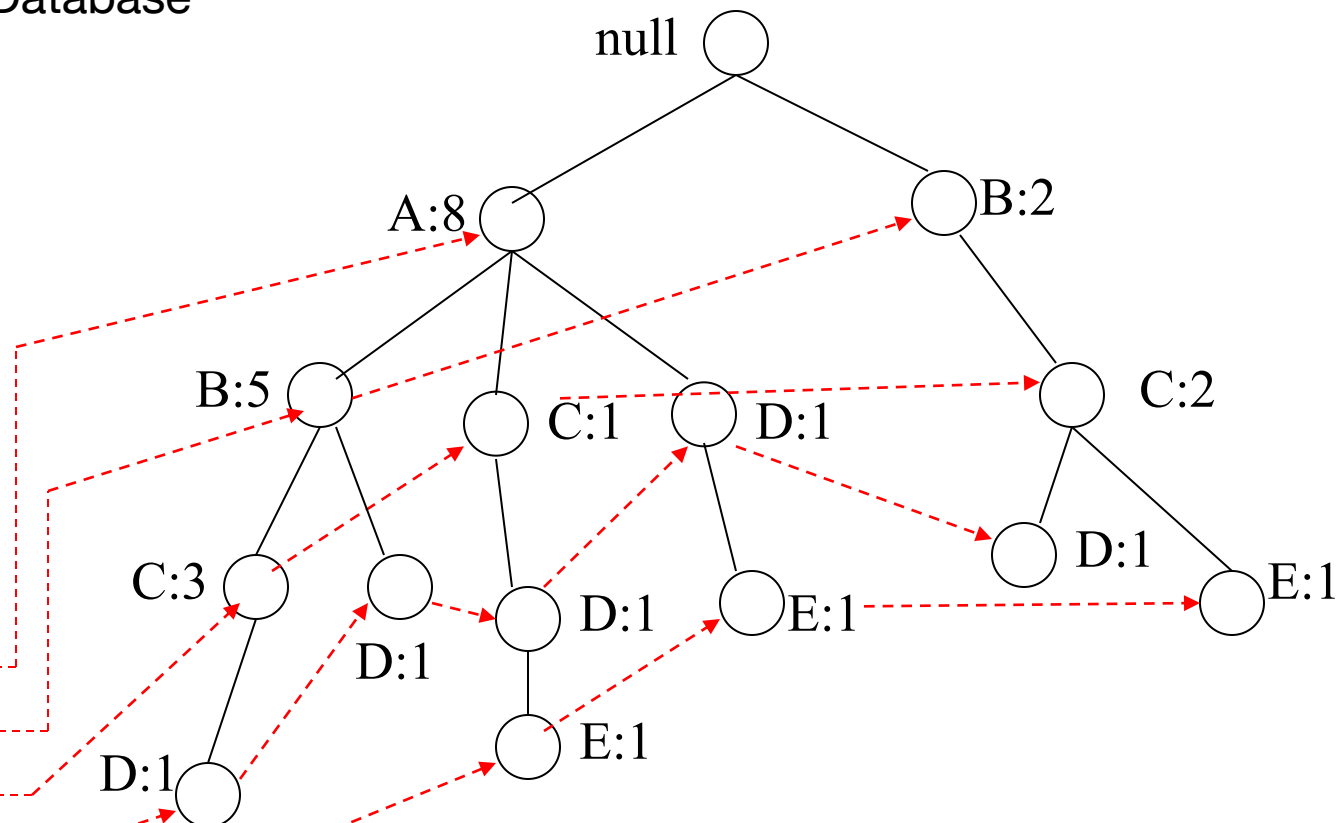
FP-Tree Construction

TID	Items
1	{A,B}
2	{B,C,D}
3	{A,C,D,E}
4	{A,D,E}
5	{A,B,C}
6	{A,B,C,D}
7	{A}
8	{A,B,C}
9	{A,B,D}
10	{B,C,E}

Transaction Database

Header table

Item	Pointer
A	
B	
C	
D	
E	



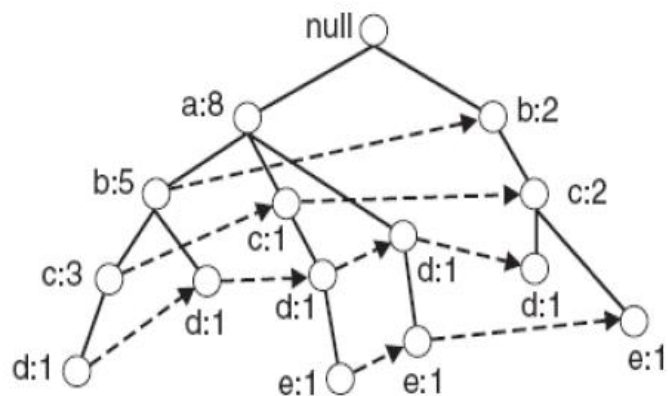
Pointers are used to assist frequent itemset generation

FP-tree Size

- The size of an FP-tree is typically smaller than **the size of the uncompressed data** because many transactions often share a few items in common
 - **Best- case** scenario: All transactions have the same set of items, and the FP-tree contains only a single branch of nodes.
 - **Worst- case** scenario: Every transaction has a unique set of items. As none of the transactions have any items in common, the size of the FP-tree is effectively the same as the size of the original data.
- The size of an FP-tree also depends on **how the items are ordered**

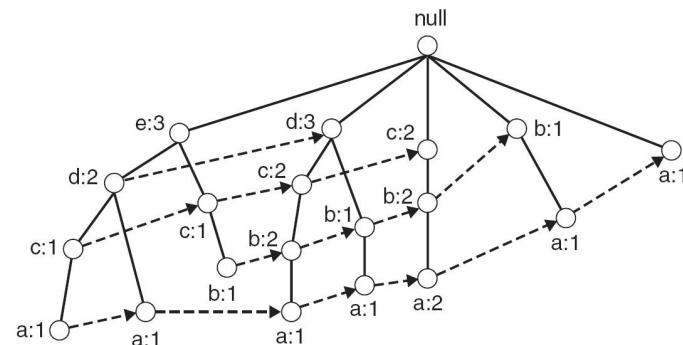
Example

- FP-tree with item descending ordering



TID	Items
1	{A,B}
2	{B,C,D}
3	{A,C,D,E}
4	{A,D,E}
5	{A,B,C}
6	{A,B,C,D}
7	{A}
8	{A,B,C}
9	{A,B,D}
10	{B,C,E}

- FP-tree with item ascending ordering



Pattern-Growth Approach

- Once an FP-tree has been constructed, it uses a **recursive divide-and-conquer approach** to mine the frequent itemsets

How to find Frequent Items?

- For each frequent item, construct its **prefix path**, and then its **conditional FP-tree**
- Repeat the process on each newly created conditional FP-tree
- Until the resulting FP-tree is empty, or it contains only one path (single path will generate all the combinations of its sub-paths, each of which is a frequent pattern)

FP-Growth Approach

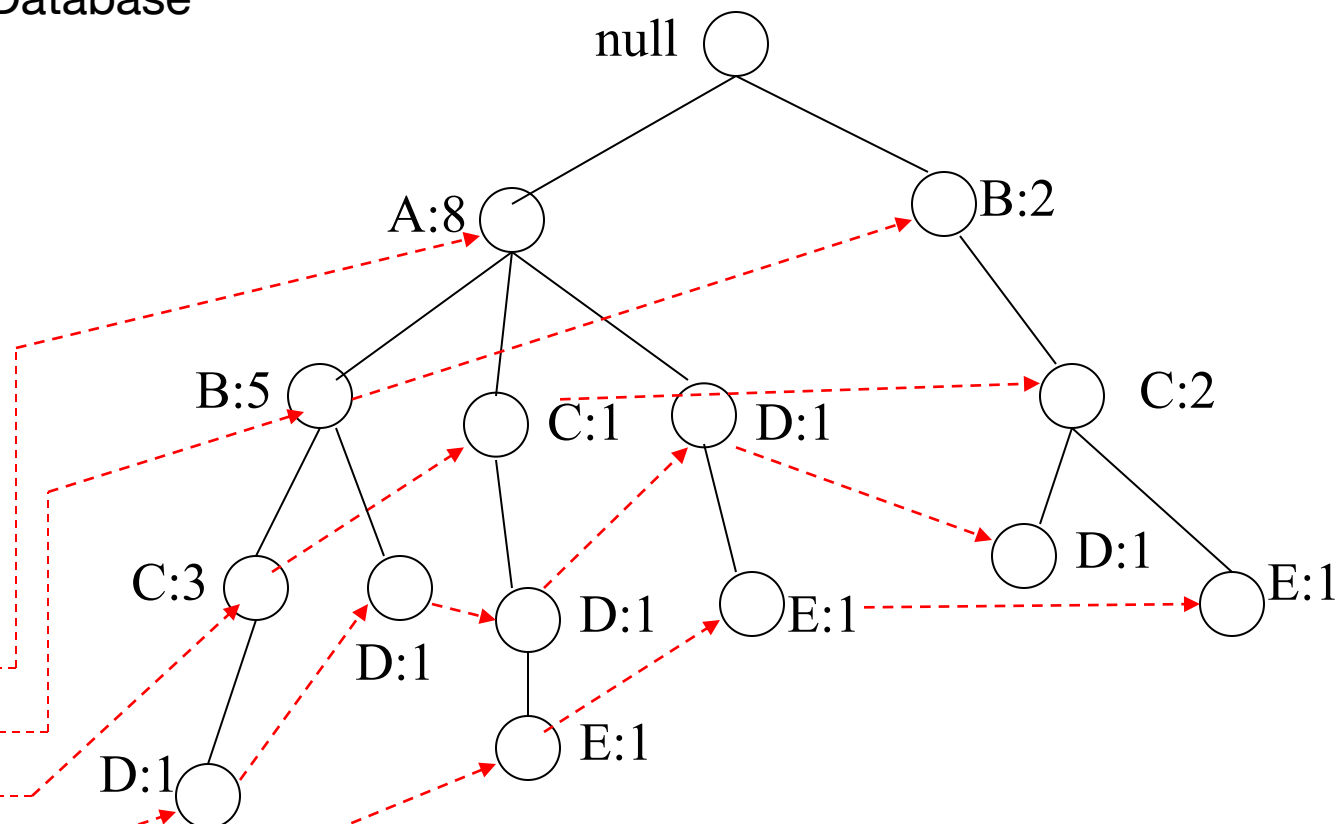
TID	Items
1	{A,B}
2	{B,C,D}
3	{A,C,D,E}
4	{A,D,E}
5	{A,B,C}
6	{A,B,C,D}
7	{A}
8	{A,B,C}
9	{A,B,D}
10	{B,C,E}

Transaction Database

Min support =2

Header table

Item	Pointer
A	
B	
C	
D	
E	

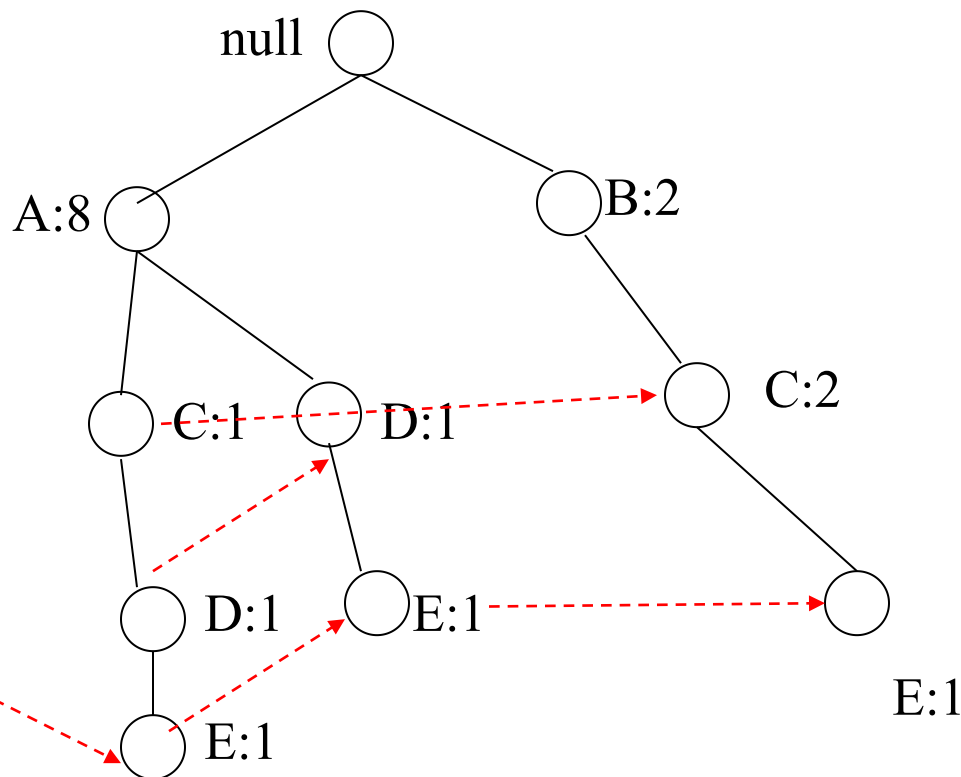


Pointers are used to assist frequent item set generation

FP-Growth Approach

Header table

Item	Pointer
A	
B	
C	
D	
E	



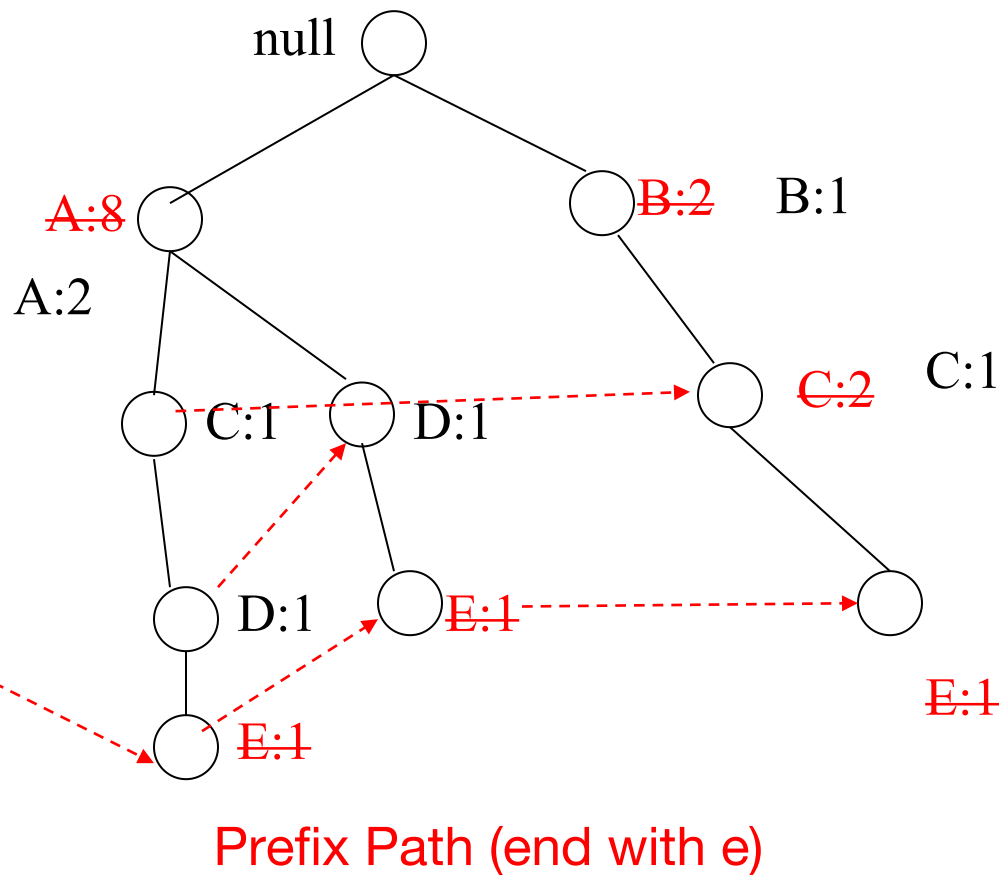
Prefix Path (end with e)

- Item e is a frequent item
- Sub problems: find frequent item end with de, ce, be, and ae
- Construct Conditional FP-Tree

FP-Growth Approach

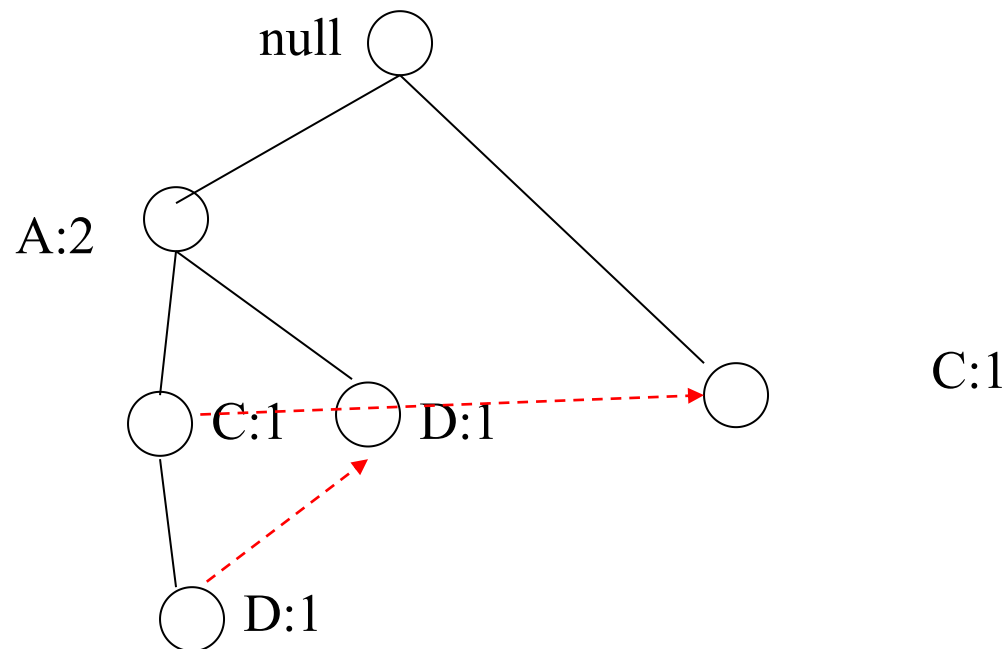
Header table

Item	Pointer
A	
B	
C	
D	
E	



- Update the support;
- Delete node e;
- Update other nodes according to their support

FP-Growth Approach



Conditional FP-Tree (end with e)

- We use this conditional FP-Tree to find frequent item end with de, ce, be and ae;
- Repeated the aforementioned steps;

- **FP-Growth:** allows frequent itemset discovery without candidate generation.
- **Two step:**
 - 1. Build a compact data structure called the FP-tree
 - 2 passes over the database
 - 2. extracts frequent itemsets directly from the FP-tree
 - Traverse through FP-tree
- Depth-first search
- Apriori: Breadth-first search



Outline

- FP-Growth
- Maximal and Closed Frequent Item Set
- Variants of Association Rule Mining
 - Mining with Multiple Minimum Supports
 - Mining Class Association Rules
 - Multi-level Association Rule
 - Sequential Pattern Mining

Compact Representation of Frequent Itemsets

- Some itemsets are redundant because they have identical support as their supersets

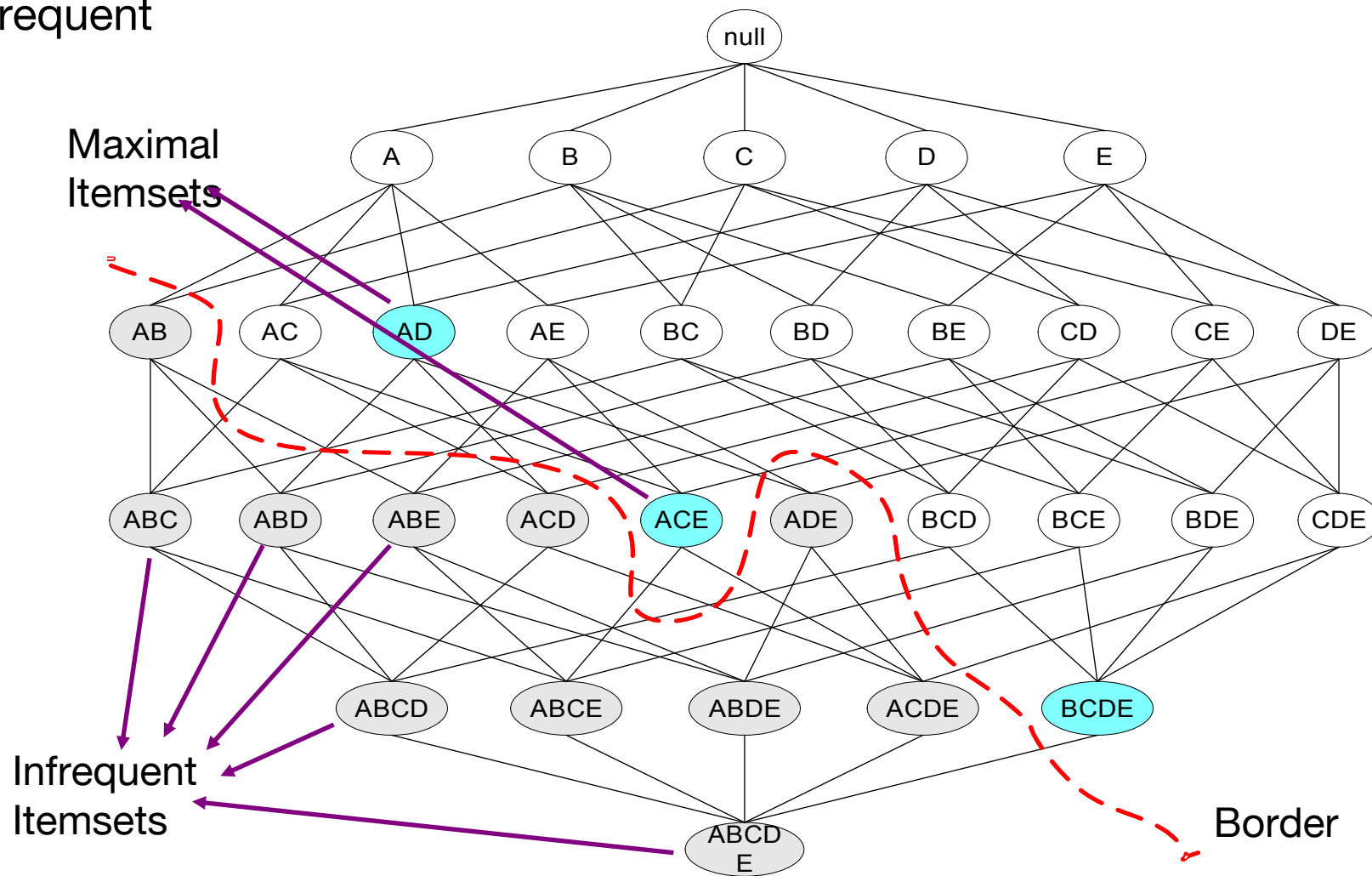
TID	A1	A2	A3	A4	A5	A6	A7	A8	A9	A10	B1	B2	B3	B4	B5	B6	B7	B8	B9	B10	C1	C2	C3	C4	C5	C6	C7	C8	C9	C10
1	1	1	1	1	1	1	1	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
2	1	1	1	1	1	1	1	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
3	1	1	1	1	1	1	1	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
4	1	1	1	1	1	1	1	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
5	1	1	1	1	1	1	1	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
6	0	0	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1	1	1	1	0	0	0	0	0	0	0	0	0
7	0	0	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1	1	1	1	0	0	0	0	0	0	0	0	0
8	0	0	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1	1	1	1	0	0	0	0	0	0	0	0	0
9	0	0	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1	1	1	1	0	0	0	0	0	0	0	0	0
10	0	0	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1	1	1	1	0	0	0	0	0	0	0	0	0
11	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1	1
12	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1	1
13	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1	1
14	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1	1
15	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1	1

MinSup=1/3

- Number of frequent itemsets $= 3 \times \sum_{k=1}^{10} \binom{10}{k}$
- Need a compact representation

Maximal Frequent Itemset

An itemset is **maximal frequent** if none of its immediate supersets is frequent



Closed Itemset

- An itemset is closed if none of its immediate supersets has the same support as the itemset

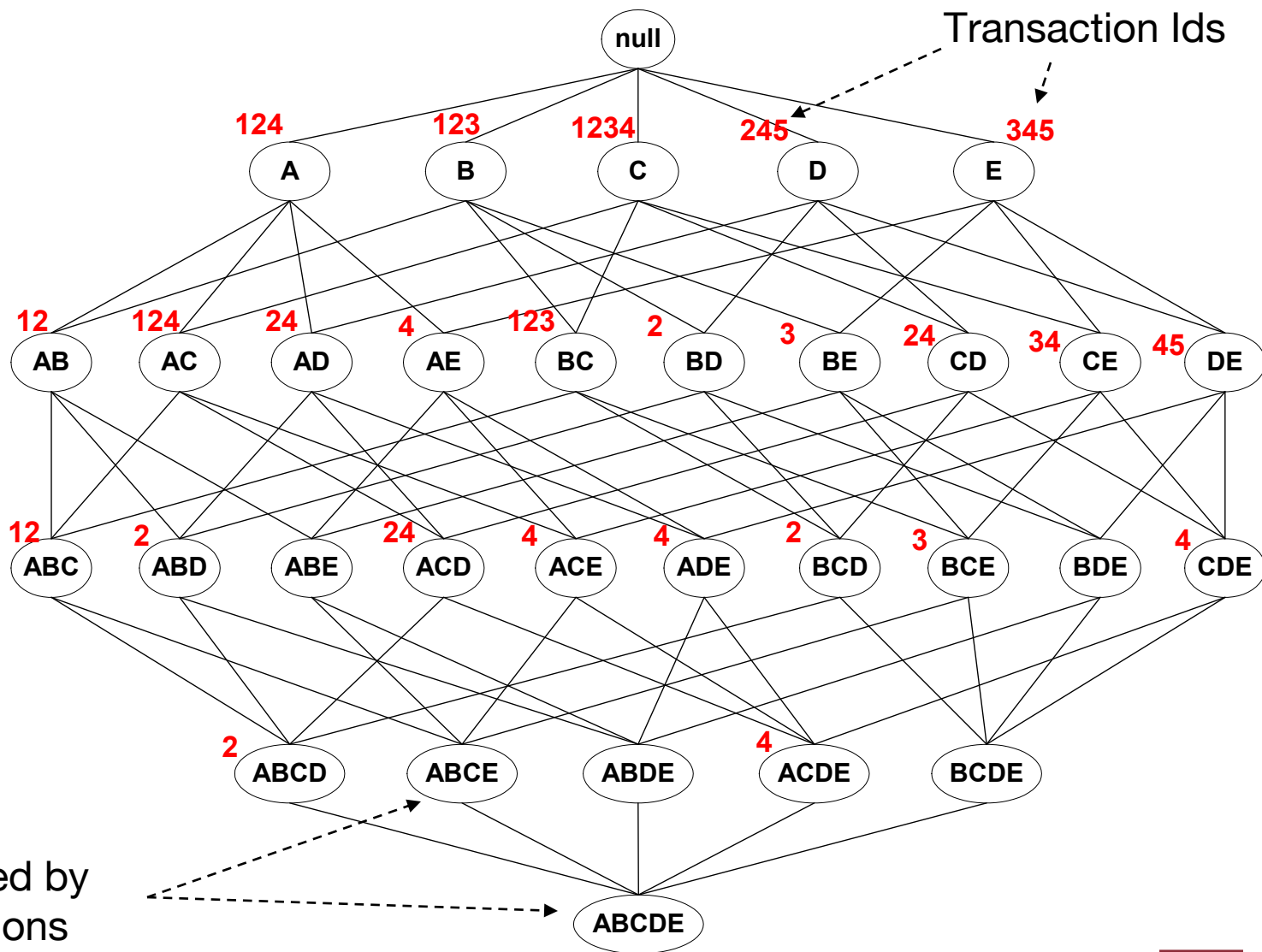
TID	Items
1	{A,B}
2	{B,C,D}
3	{A,B,C,D}
4	{A,B,D}
5	{A,B,C,D}

Itemset	Support
{A,B,C}	2
{A,B,D}	3
{A,C,D}	2
{B,C,D}	3
{A,B,C,D}	2

Itemset	Support
{A}	4
{B}	5
{C}	3
{D}	4
{A,B}	4
{A,C}	2
{A,D}	3
{B,C}	3
{B,D}	4
{C,D}	3

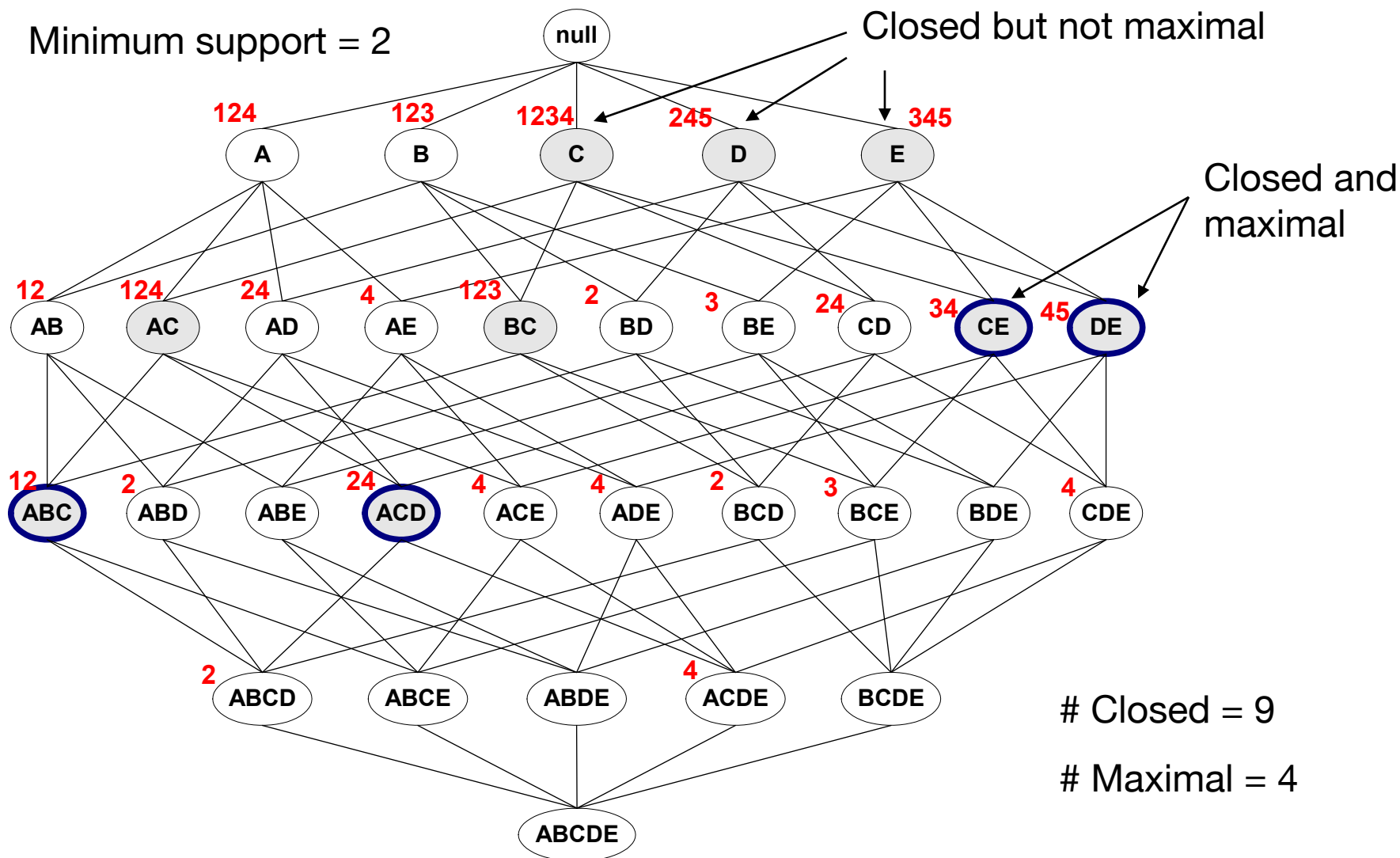
Maximal vs Closed Itemsets

TID	Items
1	ABC
2	ABCD
3	BCE
4	ACDE
5	DE

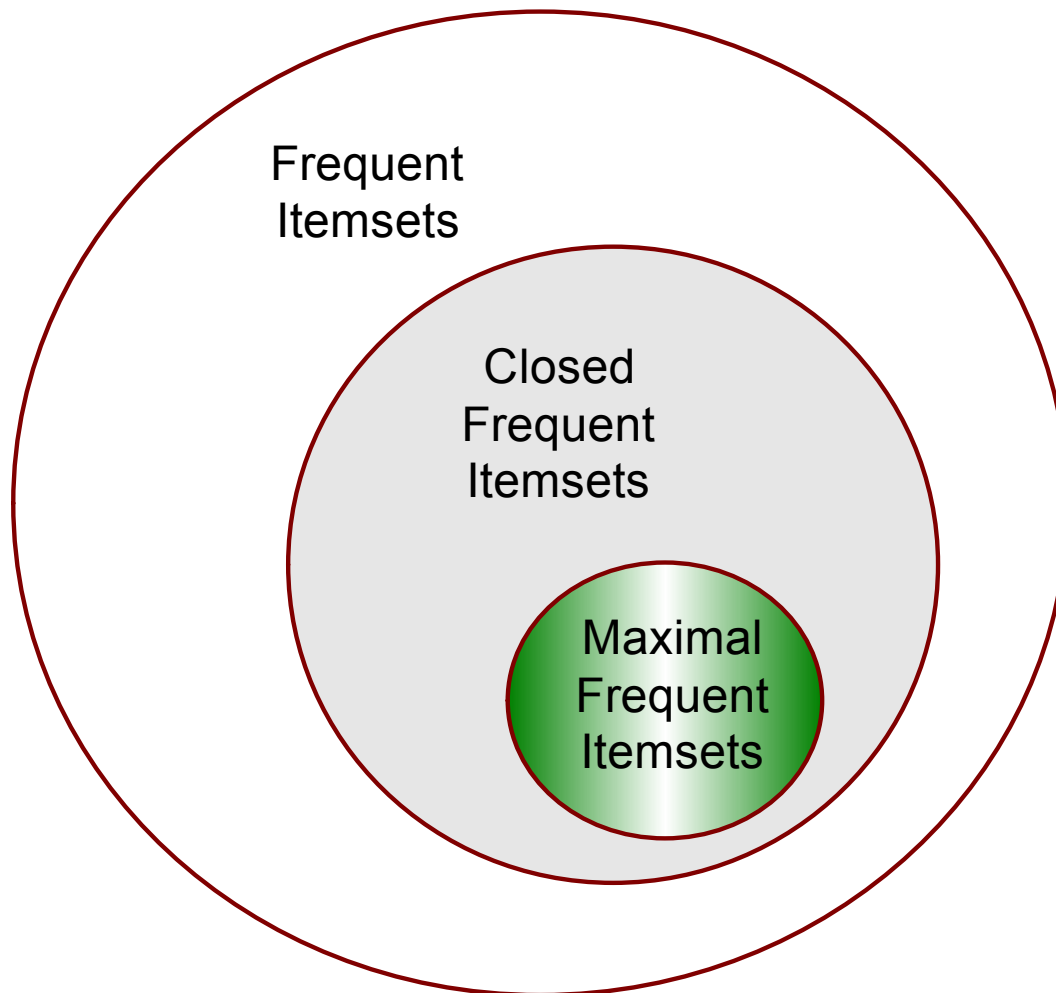


Maximal vs Closed Frequent Itemsets

Minimum support = 2



Maximal vs Closed Itemsets

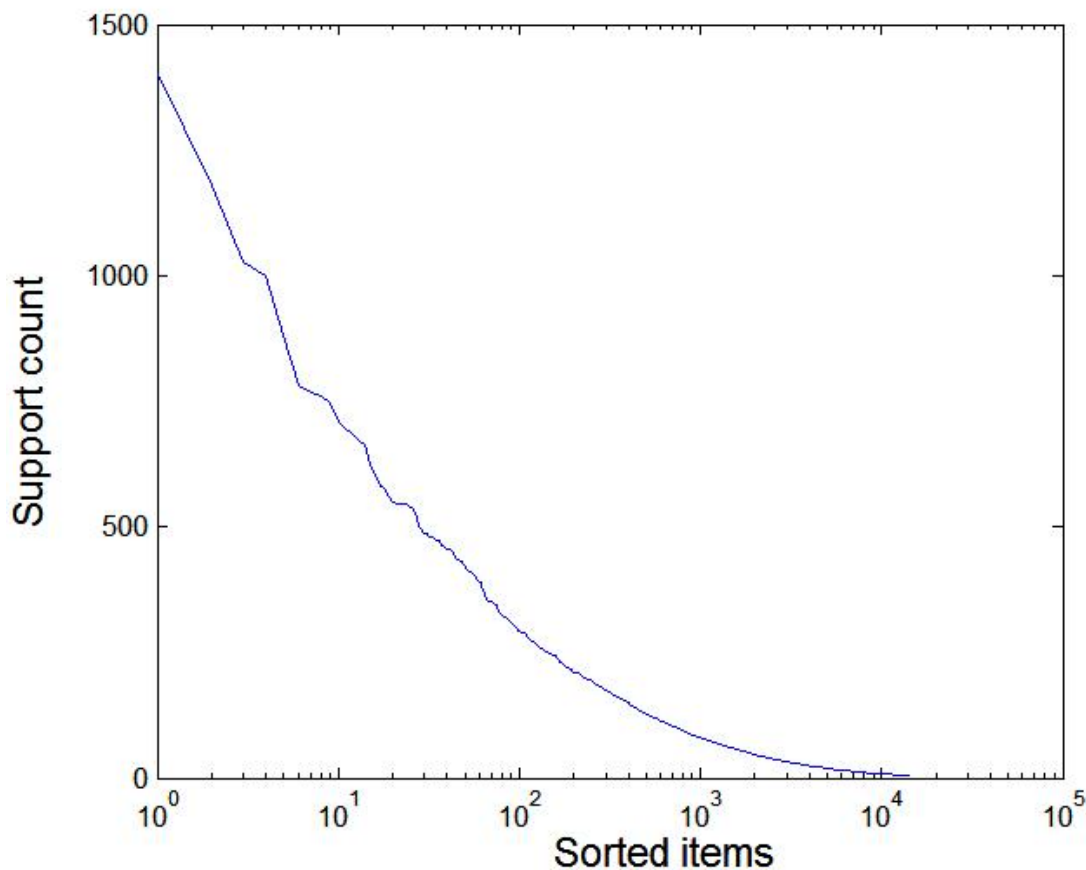


- FP-Growth
- Maximal and Closed Frequent Item set
- Variants of Association Rule Mining
 - Mining with Multiple Minimum Supports
 - Mining Class Association Rules
 - Multi-level Association Rule
 - Sequential Pattern Mining

Effect of Support Distribution

- Many real data sets have skewed support distribution

Support
distribution of a
retail data set





Effect of Support Distribution

- How to set the appropriate *minsup* threshold?
 - If *minsup* is set too high, we could miss itemsets involving interesting rare items (e.g., expensive products)
 - If *minsup* is set too low, it is computationally expensive and the number of itemsets is very large
- Using a single minimum support threshold may not be effective

Multiple Minimum Support

- How to apply multiple minimum supports?
 - $MS(i)$: minimum support for item i
 - e.g.: $MS(\text{Milk})=5\%$, $MS(\text{Coke}) = 3\%$,
 $MS(\text{Broccoli})=0.1\%$, $MS(\text{Salmon})=0.5\%$
 - $MS(\{\text{Milk}, \text{Broccoli}\}) = \min (MS(\text{Milk}), MS(\text{Broccoli}))$
 $= 0.1\%$
 - Challenge: **Support is no longer anti-monotone**
 - Suppose: $\text{Support}(\text{Milk}, \text{Coke}) = 1.5\%$ and
 $\text{Support}(\text{Milk}, \text{Coke}, \text{Broccoli}) = 0.5\%$
 - $\{\text{Milk}, \text{Coke}\}$ is infrequent but $\{\text{Milk}, \text{Coke}, \text{Broccoli}\}$ is frequent

Multiple Minimum Support (Liu 1999)

- Order the items according to their minimum support (in ascending order)
 - e.g.: $MS(\text{Milk})=5\%$, $MS(\text{Coke}) = 3\%$,
 $MS(\text{Broccoli})=0.1\%$, $MS(\text{Salmon})=0.5\%$
 - Ordering: Broccoli, Salmon, Coke, Milk
- Need to modify Apriori such that:
 - L_1 : set of frequent items
 - F_1 : set of items whose support is $\geq MS(1)$
where $MS(1)$ is $\min_i(MS(i))$
 - C_2 : candidate itemsets of size 2 is generated from F_1
instead of L_1

Multiple Minimum Support (Liu 1999)

- Modifications to Apriori:
 - In traditional Apriori,
 - A candidate $(k+1)$ -itemset is generated by merging two frequent itemsets of size k
 - The candidate is pruned if it contains any infrequent subsets of size k
 - Pruning step has to be modified:
 - Prune only if subset contains the first item
 - e.g.: Candidate={Broccoli, Coke, Milk} (ordered according to minimum support)
 - {Broccoli, Coke} and {Broccoli, Milk} are frequent but {Coke, Milk} is infrequent
 - Candidate is not pruned because {Coke, Milk} does not contain the first item, i.e., Broccoli.

- FP-Growth
- Maximal and Closed Frequent Item set
- Variants of Association Rule Mining
 - Mining with Multiple Minimum Supports
 - **Mining Class Association Rules**
 - Multi-level Association Rule
 - Sequential Pattern Mining

Mining Class Association Rules (CAR)

- Normal association rule mining does not have any target.
- It finds all possible rules that exist in data, i.e., any item can appear as a consequent or a condition of a rule.
- However, in some applications, the user is interested in some targets.
 - E.g, the user has a set of text documents from some known topics. He/she wants to find out what words are associated or correlated with each topic.

Problem Definition

- Let T be a transaction data set consisting of n transactions.
- Each transaction is also labeled with a class y .
- Let I be the set of all items in T , Y be the set of all class labels and $I \cap Y = \emptyset$.
- A **class association rule (CAR)** is an implication of the form

$$X \rightarrow y, \text{ where } X \subseteq I, \text{ and } y \in Y.$$

- The definitions of **support** and **confidence** are the same as those for normal association rules.

An Example

- **A text document data set**

doc 1:	Student, Teach, School	: Education
doc 2:	Student, School	: Education
doc 3:	Teach, School, City, Game	: Education
doc 4:	Baseball, Basketball	: Sport
doc 5:	Basketball, Player, Spectator	: Sport
doc 6:	Baseball, Coach, Game, Team	: Sport
doc 7:	Basketball, Team, City, Game	: Sport

- Let $minsup = 20\%$ and $minconf = 60\%$. The following are two examples of class association rules:

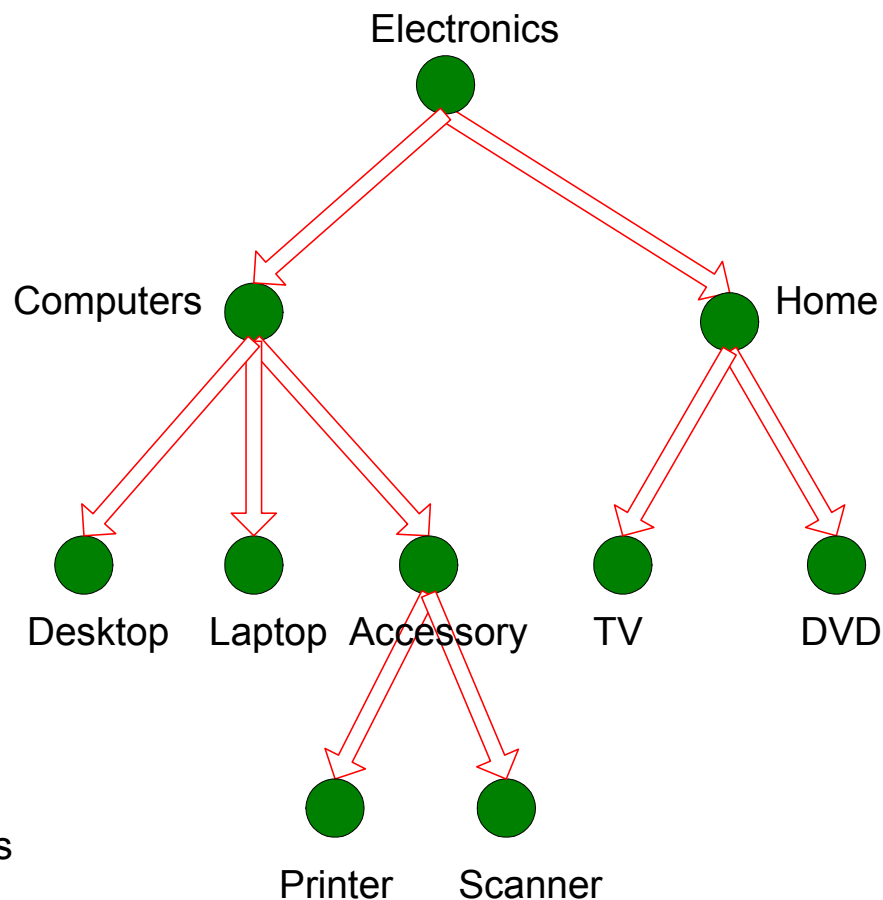
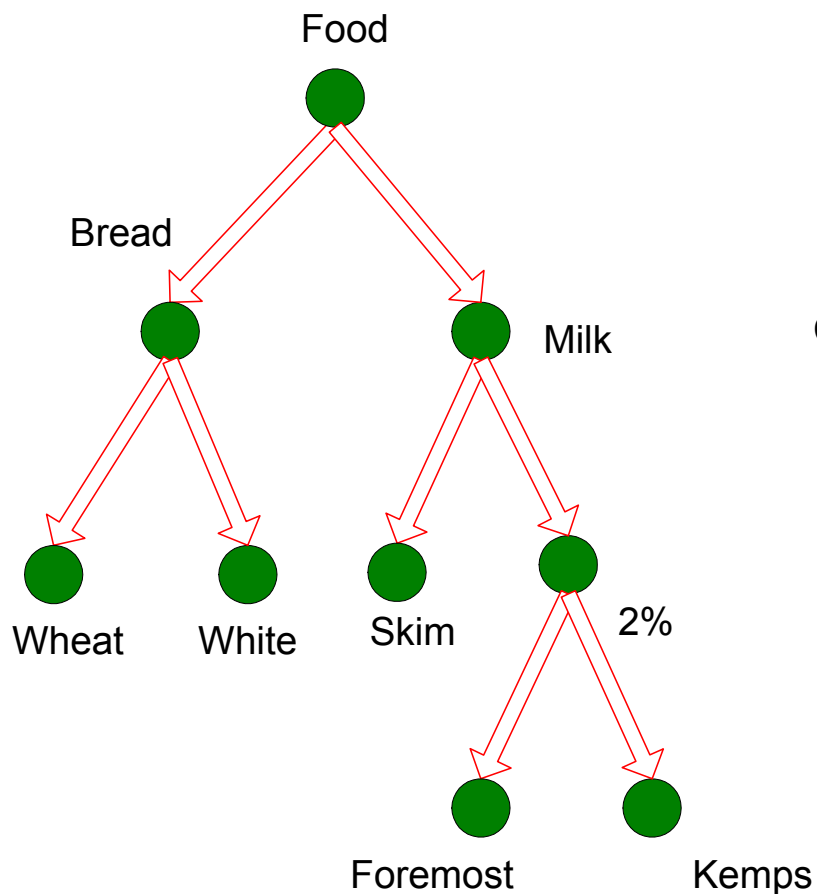
Student, School	→ Education	[sup= 2/7, conf = 2/2]
game	→ Sport	[sup= 2/7, conf = 2/3]

Mining Algorithm

- Unlike normal association rules, CARs can be mined directly in one step.
- The key operation is to find all **rule items** that have support above *minsup*. A **rule item** is of the form: **(condset, y)** where **condset** is a set of items from I (i.e., $condset \subseteq I$), and $y \in Y$ is a class label.
- Each ruleitem basically represents a rule: $condset \rightarrow y$,
- The Apriori algorithm can be modified to generate CARs

- FP-Growth
- Maximal and Closed Frequent Item Set
- Variants of Association Rule Mining
 - Mining with Multiple Minimum Supports
 - Mining Class Association Rules
 - Multi-level Association Rule
 - Sequential Pattern Mining

Multi-level Association Rules





Multi-level Association Rules

- Why should we incorporate concept hierarchy?
 - Rules at lower levels may not have enough support to appear in any frequent itemsets
 - Rules at lower levels of the hierarchy are overly specific
 - e.g., skim milk → white bread,
2% milk → wheat bread,
skim milk → wheat bread, etc.

they are indicative of association between milk and bread



Multi-level Association Rules

- Approach 1:

- Extend current association rule formulation by augmenting each transaction with higher level items

Original Transaction: {skim milk, wheat bread}

Augmented Transaction: {skim milk, wheat bread, milk, bread, food}

- Issues:

- Items that reside at higher levels have much higher support counts
 - if support threshold is low, too many frequent patterns involving items from the higher levels
- Increased dimensionality of the data



Multi-level Association Rules

- Approach 2:
 - Generate frequent patterns at highest level first
 - Then, generate frequent patterns at the next highest level, and so on
- Issues:
 - I/O requirements will increase dramatically because we need to perform more passes over the data
 - May miss some potentially interesting cross-level association patterns

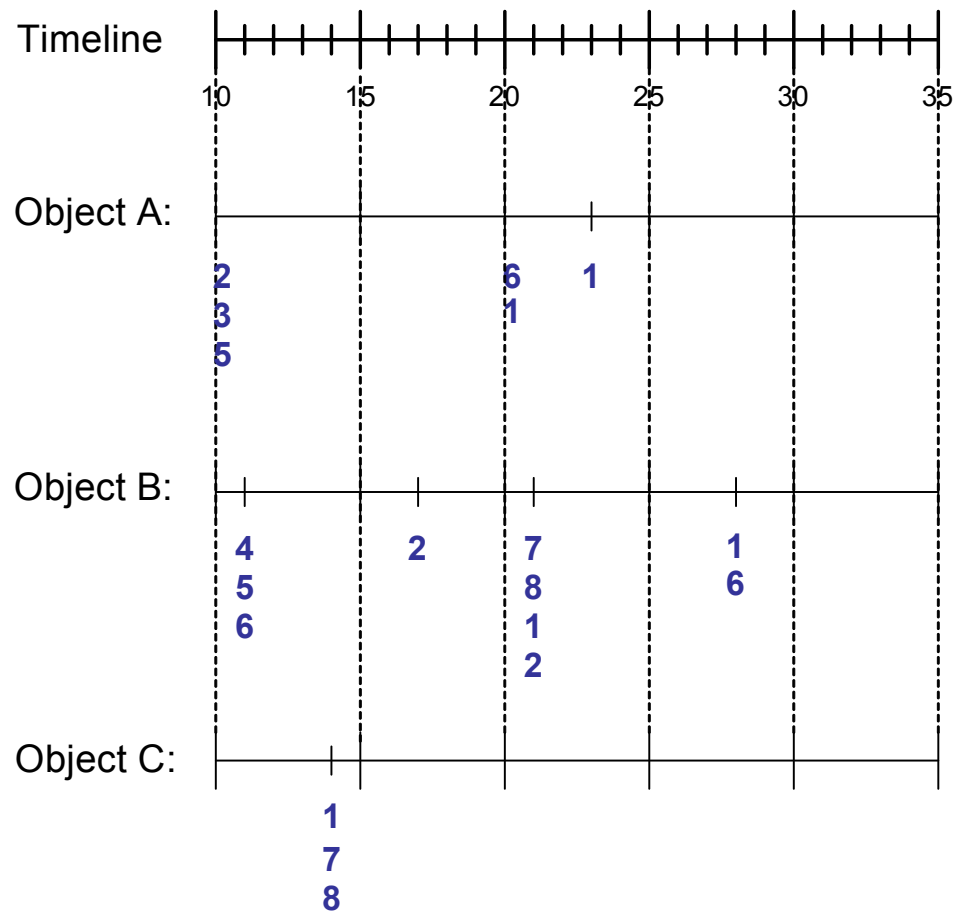
Sequence Data

Sequence Database	Sequence
Customer	Purchase history of a given customer
Web Data	Browsing activity of a particular Web visitor
Event data	History of events generated by a given sensor
Genome sequences	DNA sequence of a particular species
...	...

Sequence Data

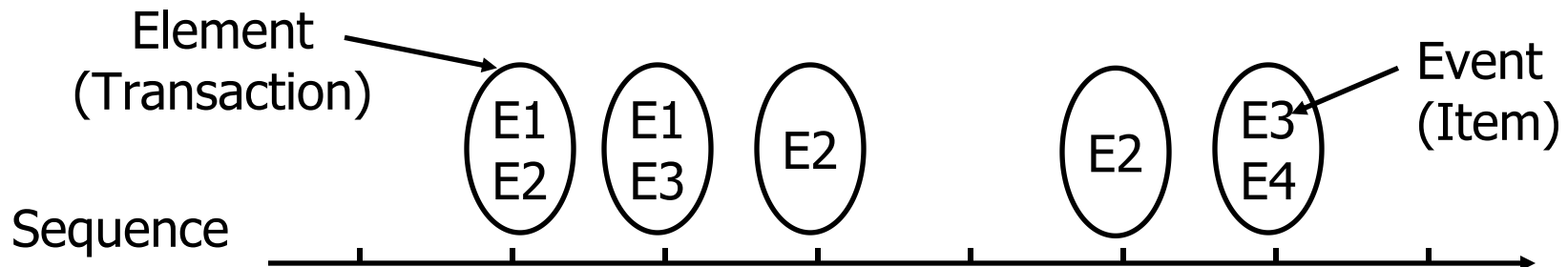
Sequence Database:

Object	Timestamp	Events
A	10	2, 3, 5
A	20	6, 1
A	23	1
B	11	4, 5, 6
B	17	2
B	21	7, 8, 1, 2
B	28	1, 6
C	14	1, 8, 7



Examples of Sequence Data

Sequence Database	Sequence	Element (Transaction)	Event (Item)
Customer	Purchase history of a given customer	A set of items bought by a customer at time t	Books, diary products, CDs, etc
Web Data	Browsing activity of a particular Web visitor	A collection of files viewed by a Web visitor after a single mouse click	Home page, index page, contact info, etc
Event data	History of events generated by a given sensor	Events triggered by a sensor at time t	Types of alarms generated by sensors
Genome sequences	DNA sequence of a particular species	An element of the DNA sequence	Bases A,T,G,C



Definition of a Sequence

- A sequence is an ordered list of **elements** (transactions)

$$s = \langle e_1 e_2 e_3 \dots \rangle$$

- Each element contains a collection of **events** (items)

$$e_i = \{i_1, i_2, \dots, i_k\}$$

- Each element is attributed to a specific time or location

- **Length of a sequence**, $|s|$, is given by the number of elements of the sequence
- A **k-sequence** is a sequence that contains **k events** (items)

Definition of a Subsequence

- A sequence $\langle a_1 a_2 \dots a_n \rangle$ is contained in another sequence $\langle b_1 b_2 \dots b_m \rangle$ ($m \geq n$) if there exist integers $i_1 < i_2 < \dots < i_n$ such that $a_1 \subseteq b_{i_1}$, $a_2 \subseteq b_{i_2}$, ..., $a_n \subseteq b_{i_n}$

Data sequence	Subsequence	Contain?
$\langle \{2,4\} \{3,5,6\} \{8\} \rangle$	$\langle \{2\} \{3,5\} \rangle$	Yes
$\langle \{1,2\} \{3,4\} \rangle$	$\langle \{1\} \{2\} \rangle$	No
$\langle \{2,4\} \{2,4\} \{2,5\} \rangle$	$\langle \{2\} \{4\} \rangle$	Yes

- The support of a subsequence w is defined as the fraction of data sequences that contain w
- A sequential pattern is a frequent subsequence** (i.e., a subsequence whose support is $\geq \text{minsup}$)

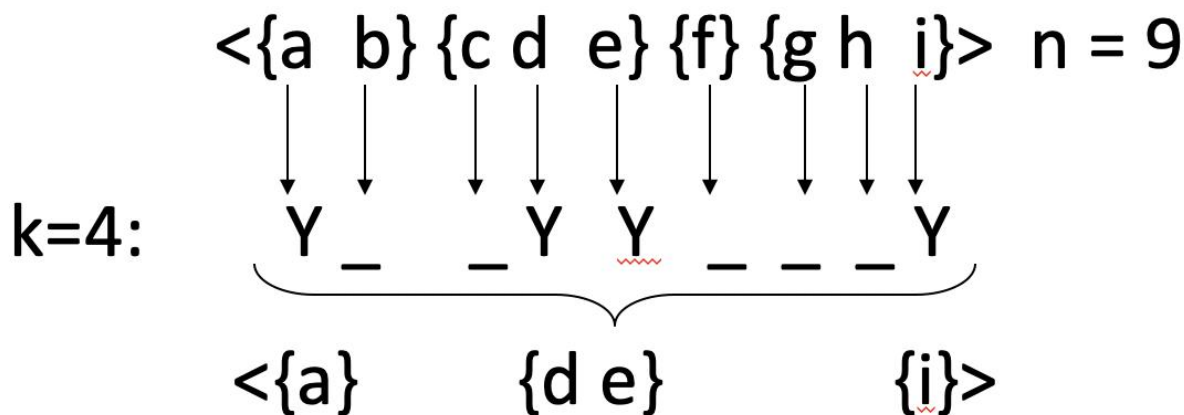


Sequential Pattern Mining: Definition

- Given:
 - a database of sequences
 - a user-specified minimum support threshold, *minsup*
- Task:
 - Find all subsequences with support $\geq \textit{minsup}$

Sequential Pattern Mining: Challenge

- Given a sequence: $\langle \{a\ b\} \{c\ d\ e\} \{f\} \{g\ h\ i\} \rangle$
 - Examples of subsequences:
 $\langle \{a\} \{c\ d\} \{f\} \{g\} \rangle$, $\langle \{c\ d\ e\} \rangle$, $\langle \{b\} \{g\} \rangle$, etc.
- How many k -subsequences can be extracted from a given n -sequence?



Answer :

$$\binom{n}{k} = \binom{9}{4} = 126$$

Sequential Pattern Mining: Example

Object	Timestamp	Events
A	1	1,2,4
A	2	2,3
A	3	5
B	1	1,2
B	2	2,3,4
C	1	1, 2
C	2	2,3,4
C	3	2,4,5
D	1	2
D	2	3, 4
D	3	4, 5
E	1	1, 3
E	2	2, 4, 5

Minsup = 50%

Examples of Frequent Subsequences:

< {1,2} >	s=60%
< {2,3} >	s=60%
< {2,4}>	s=80%
< {3} {5}>	s=80%
< {1} {2} >	s=80%
< {2} {2} >	s=60%
< {1} {2,3} >	s=60%
< {2} {2,3} >	s=60%
< {1,2} {2,3} >	s=60%

Extracting Sequential Patterns

- Given n events: $i_1, i_2, i_3, \dots, i_n$
- Candidate 1-subsequences:
 $\langle \{i_1\} \rangle, \langle \{i_2\} \rangle, \langle \{i_3\} \rangle, \dots, \langle \{i_n\} \rangle$
- Candidate 2-subsequences:
 $\langle \{i_1, i_2\} \rangle, \langle \{i_1, i_3\} \rangle, \dots, \langle \{i_1\} \{i_1\} \rangle, \langle \{i_1\} \{i_2\} \rangle, \dots, \langle \{i_{n-1}\} \{i_n\} \rangle$
- Candidate 3-subsequences:
 $\langle \{i_1, i_2, i_3\} \rangle, \langle \{i_1, i_2, i_4\} \rangle, \dots, \langle \{i_1, i_2\} \{i_1\} \rangle, \langle \{i_1, i_2\} \{i_2\} \rangle, \dots,$
 $\langle \{i_1\} \{i_1, i_2\} \rangle, \langle \{i_1\} \{i_1, i_3\} \rangle, \dots, \langle \{i_1\} \{i_1\} \{i_1\} \rangle, \langle \{i_1\} \{i_1\} \{i_2\} \rangle, \dots$

Generalized Sequential Pattern (GSP)

- **Step 1:**
 - Make the first pass over the sequence database D to yield all the 1-element frequent sequences
- **Step 2:**

Repeat until no new frequent sequences are found

 - **Candidate Generation:**
 - Merge pairs of frequent subsequences found in the $(k-1)th$ pass to generate candidate sequences that contain k items
 - **Candidate Pruning:**
 - Prune candidate k -sequences that contain infrequent $(k-1)$ -subsequences
 - **Support Counting:**
 - Make a new pass over the sequence database D to find the support for these candidate sequences
 - **Candidate Elimination:**
 - Eliminate candidate k -sequences whose actual support is less than *minsup*

GSP Example

Frequent 3-sequences

$\langle \{1\} \{2\} \{3\} \rangle$
 $\langle \{1\} \{2\ 5\} \rangle$
 $\langle \{1\} \{5\} \{3\} \rangle$
 $\langle \{2\} \{3\} \{4\} \rangle$
 $\langle \{2\ 5\} \{3\} \rangle$
 $\langle \{3\} \{4\} \{5\} \rangle$
 $\langle \{5\} \{3\ 4\} \rangle$

Candidate Generation

$\langle \{1\} \{2\} \{3\} \{4\} \rangle$
 $\langle \{1\} \{2\ 5\} \{3\} \rangle$
 $\langle \{1\} \{5\} \{3\ 4\} \rangle$
 $\langle \{2\} \{3\} \{4\} \{5\} \rangle$
 $\langle \{2\ 5\} \{3\ 4\} \rangle$

Candidate Pruning

$\langle \{1\} \{2\ 5\} \{3\} \rangle$

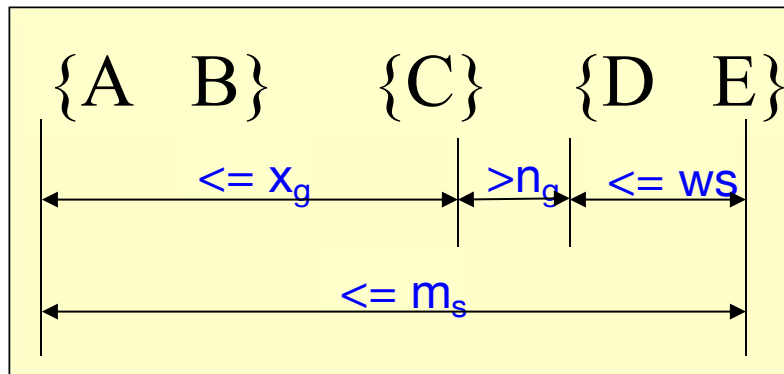
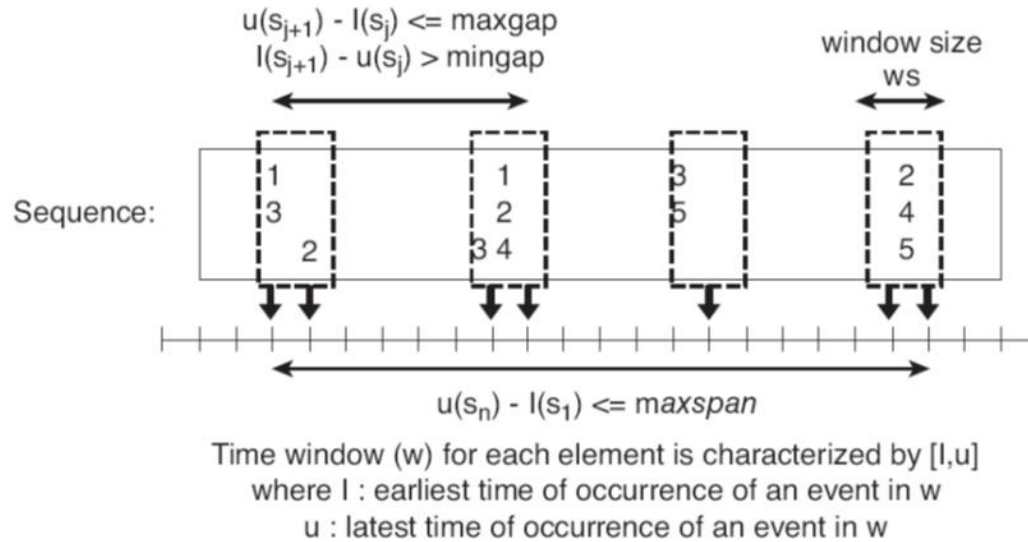
Constraints

- Item constraint
 - Find web log patterns only about online-bookstores
- Length constraint
 - Find patterns having at least 20 items
- Super pattern constraint
 - Find super patterns of “PC → digital camera”
- Aggregate constraint
 - Find patterns that the average price of items is over \$100

Constraints

- Regular expression constraint
 - Find patterns “starting from Yahoo homepage, search for hotels in Washington DC area”
 - `Yahootravel(WashingtonDC|DC)(hotel|motel|lodging)`
- Duration constraint
 - Find patterns about ± 24 hours of a shooting
- Gap constraint
 - Find purchasing patterns such that “the gap between each consecutive purchases is less than 1 month”

Timing Constraints



x_g : max-gap

n_g : min-gap

ws: window size

m_s : maximum span



With Timing Constraints

- Approach 1:
 - Mine sequential patterns without timing constraints
 - Postprocess the discovered patterns
- Approach 2:
 - Modify GSP to directly prune candidates that violate timing constraints

Existing Methods

- Concept introduction and an initial Apriori-like algorithm
 - Agrawal & Srikant. Mining sequential patterns, [ICDE'95]
- Apriori-based method: **GSP** (Generalized Sequential Patterns: Srikant & Agrawal [EDBT'96])
- Pattern-growth methods: FreeSpan & **PrefixSpan** (Han et al. KDD'00; Pei, et al. [ICDE'01])
- Vertical format-based mining: **SPADE** (Zaki [Machine Learning'00])
- Constraint-based sequential pattern mining (SPIRIT: Garofalakis, Rastogi, Shim [VLDB'99]; Pei, Han, Wang [CIKM'02])
- Mining closed sequential patterns: **CloSpan** (Yan, Han & Afshar [SDM'03])



Some Takeaways

- FP-Growth
- Maximal and Closed Frequent Item set
- Variants of Association Rule Mining

Acknowledgement

- Introduction to data mining (by Tan, Steinbach, Kumar)
- <https://mran.microsoft.com/snapshot/2014-11-17/web/packages/TraMineR/TraMineR.pdf>
- <http://traminer.unige.ch/contrib.shtml>

The End

■ THANK YOU

Any questions?

